STATECRAFT ONLINE

Dashboard and Strategic Map

Implementation reference optimized for human review and machine extraction

Document purpose

This document explains how the React frontend displays the primary nation dashboard and the strategic map. It identifies routes, components, API dependencies, state transitions, realtime refresh behavior, presentation structure, and responsive CSS. The source paths and code excerpts are preserved as plain text so an AI system can reason about the implementation without interpreting screenshots.

System summary

Concern	Implementation
Dashboard route	/nation/:id renders NationProfilePage
Map route	/nation/:id/map renders MapPage
Shared navigation	NationNav preserves the active nation ID across nation sections
Dashboard data	Nation profile plus owner development data
Map data	Nation identity, map locations, and owner development data
Map rendering	Fixed 10 x 10 semantic HTML grid; location cells are buttons
Realtime refresh	Nation turn, development, and technology messages trigger refetches
Responsive behavior	Multi-column layouts collapse to one column below 880 px

Primary source files

File	Responsibility
apps/web/src/App.tsx	Declares dashboard and map routes.
apps/web/src/pages/NationProfilePage.tsx	Loads and renders the main nation dashboard.
apps/web/src/pages/MapPage.tsx	Loads map state and renders the map/detail layout.
apps/web/src/components/MapGrid.tsx	Maps x/y locations onto a 10 x 10 grid.
apps/web/src/components/StatGrid.tsx	Renders eight national indicators with percentage bars.
apps/web/src/components/NationNav.tsx	Provides navigation among nation systems.
apps/web/src/styles.css	Controls panel grids, map cells, selection, and responsive layouts.

1. Routing and Navigation

React Router binds the nation identifier from the URL to each page. The same NationNav component is included in both screens, keeping links scoped to the active nation.

```
// apps/web/src/App.tsx
<Routes>
  <Route path="/nation/:id" element={<NationProfilePage />} />
  <Route path="/nation/:id/profile" element={<NationProfilePage />} />
  <Route path="/nation/:id/map" element={<MapPage />} />
</Routes>

// apps/web/src/components/NationNav.tsx
<nav className="nation-nav" aria-label="Nation sections">
  <NavLink end to={`/nation/${nationId}`}>Dashboard</NavLink>
  <NavLink to={`/nation/${nationId}/feed`}>World Feed</NavLink>
  <NavLink to={`/nation/${nationId}/news`}>News</NavLink>
  <NavLink to={`/nation/${nationId}/events`}>Events</NavLink>
  <NavLink to={`/nation/${nationId}/technology`}>Technology</NavLink>
  <NavLink to={`/nation/${nationId}/map`}>Map</NavLink>
  <NavLink to={`/nation/${nationId}/development`}>Development</NavLink>
  <NavLink to={`/nation/${nationId}/agents`}>Agents</NavLink>
  <NavLink to={`/nation/${nationId}/military`}>Military</NavLink>
</nav>
```

Route behavior

The dashboard is both the canonical /nation/:id page and the /profile alias. NavLink uses the end flag for Dashboard so that it is not marked active on every nested nation route. All navigation URLs interpolate nationId, avoiding a hard-coded demo nation.

2. Main Dashboard

Component: NationProfilePage

The page obtains id from useParams, loads a public profile, and attempts to load owner-only development data. Development failure is converted to null so the public profile can still render.

```
const { id } = useParams();
const nationId = id ?? "";

useEffect(() => {
  Promise.all([
    getNationProfile(nationId),
    getNationDevelopment(nationId).catch(() => null)
  ])
    .then(([loadedProfile, loadedDevelopment]) => {
      setProfile(loadedProfile);
      setDevelopment(loadedDevelopment);
    })
    .catch((caught: Error) => setError(caught.message));
}, [nationId]);
```

Dashboard profile payload

Property	Displayed purpose
nation	Name, motto, turn, description, capital, government, economy type, origin, traits, flag identity
stats	Economy, stability, liberty, authority, military, technology, environment, public trust
economy	Treasury, population, and resource balances
activeEvents	The first unresolved national issue
eventHistory	The most recent resolved issue

Property	Displayed purpose
recentPosts	Loaded in the profile contract; not directly rendered in the shown dashboard body
importantMapLocations	Location summary list
agentsSummary	Agent name and role summary
militarySummary	Military unit name and type summary
ideologySummary	Readable political-character statements

Dashboard Presentation Structure

```
<main className="page-shell">
  <header className="page-header">
    <div>
      <p className="eyebrow">Nation Profile</p>
      <h1>{profile.nation.name}</h1>
    </div>
    <NationNav nationId={profile.nation.id} />
  </header>

  <section className="profile-layout">
    <article className="panel">Identity and flag</article>
    <article className="panel">Development projects and slots</article>
    <article className="panel">
      <StatGrid stats={profile.stats} />
    </article>
  </section>

  <section className="two-column">
    <article className="panel">Economy and resources</article>
    <article className="panel">Ideology</article>
    <article className="panel">Current issue</article>
    <article className="panel">Recent resolution</article>
  </section>

  <section className="three-column-summary">
    <article className="panel">Locations</article>
    <article className="panel">Agents</article>
    <article className="panel">Military</article>
  </section>
</main>
```

National indicators

StatGrid iterates over a fixed ordered list of eight stat keys. Each row renders a formatted label, numeric value, and bar whose inline width equals the stat percentage.

```
const statKeys = [
  "economy", "stability", "liberty", "authority",
  "military", "technology", "environment", "publicTrust"
];

<div className="stat-grid">
  {statKeys.map((key) => (
    <div className="stat-row" key={key}>
      <span>{formatEnum(key)}</span>
      <strong>{stats[key]}</strong>
      <div className="stat-bar" aria-hidden="true">
        <div style={{ width: `${stats[key]}%` }} />
      </div>
    </div>
  ))}
</div>
```

Realtime dashboard refresh

The dashboard subscribes to nation turn advancement, location upgrade lifecycle, and technology lifecycle messages. When the message nationId matches the displayed nation, it refetches profile and development state. This favors consistency over fine-grained client-side patching.

3. Strategic Map

Component: MapPage

The map page maintains nationName, locations, development, selectedLocation, loading, and error state. Its refresh function requests all required datasets concurrently and preserves the current selection when possible.

```
const refresh = useCallback(async () => {
  const [nation, loadedLocations, loadedDevelopment] = await Promise.all([
    getNation(nationId),
    getMapLocations(nationId),
    getNationDevelopment(nationId).catch(() => null)
  ]);

  setNationName(nation.name);
  setLocations(loadedLocations);
  setSelectedLocation((current) =>
    loadedLocations.find((item) => item.id === current?.id)
      ?? loadedLocations[0]
      ?? null
  );
  setDevelopment(loadedDevelopment);
  setLoaded(true);
}, [nationId]);
```

Map Rendering

MapGrid is a presentation component. It receives locations, a selected ID, and an onSelect callback. It does not fetch data or own gameplay rules.

```
const width = 10;
const height = 10;

const cells = Array.from({ length: width * height }, (_, index) => {
  const x = index % width;
  const y = Math.floor(index / width);
  const location = locations.find(
    (item) => item.x === x && item.y === y
  );

  return location ? (
    <button
      className={`map-cell map-cell--location ${
        location.id === selectedLocationId ? "map-cell--selected" : ""
      }`}
      key={`${x}-${y}`}
      type="button"
      onClick={() => onSelect(location)}
      aria-label={`${location.name}, ${formatEnum(location.type)}`}
    >
      <span className="map-marker">{locationMarker(location.type)}</span>
      <span className="map-label">{location.name}</span>
    </button>
  ) : (
    <div className="map-cell" key={`${x}-${y}`} aria-hidden="true" />
  );
});

return <div className="map-grid" role="grid">{cells}</div>;
```

Map interaction model

Action or state	Behavior
Initial load	Selects the first available location.
Grid location click	Calls onSelect(location) and updates the detail panel.
Location-list click	Provides a second, text-forward way to select the same location.
Empty grid cell	Rendered as a noninteractive aria-hidden div.
Selected cell	Receives map-cell--selected and a gold inset outline.
Owner development available	Adds yield, assigned agents, and construction details.
Owner development unavailable	The public map still renders basic location information.
Realtime project/turn event	Refetches nation, location, and development data.

Selected-location detail

The side panel displays location type, coordinates, resource type, population, and development level. When development data is available, it also displays treasury output, upkeep, assigned agents, and any active upgrade project's target level and completion turn.

4. CSS and Responsive Layout

```
.page-shell {
  width: min(1180px, calc(100% - 2rem));
```

```
  margin: 0 auto;
  padding: 2rem 0 4rem;
}

.two-column,
.map-layout {
  display: grid;
  grid-template-columns: minmax(0, 1fr) minmax(280px, 0.42fr);
  gap: 1rem;
}

.map-grid {
  display: grid;
  grid-template-columns: repeat(10, minmax(0, 1fr));
  gap: 0.35rem;
  width: 100%;
  aspect-ratio: 1 / 1;
  min-height: 360px;
}

.map-cell--selected {
  outline: 2px solid #f0c96d;
  outline-offset: -2px;
}
```

At widths below 880 px, the page header becomes vertical and dashboard/map multi-column layouts collapse to a single column. Below 560 px, the map gap shrinks, minimum height becomes 300 px, and text labels inside cells are hidden while accessible button labels remain.

5. End-to-End Data Flow

Step	Dashboard	Map
1. URL	/nation/:id	/nation/:id/map
2. Route component	NationProfilePage	MapPage
3. Main API calls	getNationProfile; getNationDevelopment	getNation; getMapLocations; getNationDevelopment
4. Local state	profile; development; error	name; locations; development; selection; loaded; error
5. Child components	NationNav; FlagPreview; StatGrid	NationNav; MapGrid
6. User interaction	Navigate to systems and open issue/development	Select location and open development management
7. Realtime strategy	Refetch matching nation data	Refetch matching nation data

Machine-oriented interpretation

The implementation follows a route-page-component pattern. Route pages own API loading, error/loading state, realtime invalidation, and composition. Small components own reusable presentation. Shared domain types define API-shaped state. Gameplay calculations are not performed in these React components; the frontend displays server-authoritative profile, development, location, and economy information.

Notable implementation characteristics

1. The current map is a logical grid, not canvas, SVG, WebGL, or a geographic projection.
2. Coordinates must fit the 0-9 x and 0-9 y grid to appear.
3. Location lookup is performed with Array.find for each of 100 cells, which is acceptable at the current scale.
4. Public rendering degrades gracefully when owner-only development data cannot be loaded.
5. Realtime handling uses full refetches, reducing stale-state complexity.
6. The dashboard name NationProfilePage is historical; it currently functions as the main nation dashboard.

Source references

All paths are relative to D:/GameDev/NationRealms.
- apps/web/src/App.tsx
- apps/web/src/pages/NationProfilePage.tsx
- apps/web/src/pages/MapPage.tsx
- apps/web/src/components/MapGrid.tsx
- apps/web/src/components/StatGrid.tsx
- apps/web/src/components/NationNav.tsx
- apps/web/src/styles.css

Generated from the repository implementation inspected on 2026-07-12.